

# Job-Marketplace-Applikation

04.09.2026

---

## Änderungen gegenüber V1.0

Version 1.0 beschreibt das in Modul 295 erstellte Backend. Version 2.0 ergänzt die Dokumentation um das in Modul 294 entwickelte Angular-Frontend und führt die Kapitel nach, die von den dafür nötigen Backend-Anpassungen betroffen sind.

Neu hinzugekommen sind:

1. das Hauptkapitel „Frontend“ mit Wireframes, Komponentenübersicht, Routing, Services, Berechtigungen und einer Kurzanleitung
2. der zusätzliche Keycloak-Client für die Angular-Anwendung
3. der Endpoint GET /api/users/me

Geändert wurden:

- Die Create-Endpoints nehmen den handelnden Benutzer nicht mehr aus dem Request-Body, sondern aus dem Access-Token.
- CSRF-Schutz ist aktiviert.
- Die Registrierung meldet Fehler über aussagekräftige HTTP-Statuscodes.

Da das Backend angepasst wurde, werden beide Projekte abgegeben.

---

## Ausgangslage und Idee

Im Rahmen dieser Projektarbeit wird ein Backend für eine Job-Marketplace-Applikation entwickelt. Die Anwendung dient als Plattform, auf welcher Arbeitgeber Stelleninserate erstellen und verwalten können, während Arbeitssuchende diese Stellen einsehen, speichern und sich darauf bewerben können.

Das Ziel des Projekts ist es, eine vollständige REST-API mit Spring Boot zu entwickeln, welche alle grundlegenden Funktionen eines einfachen Jobportals abdeckt. Dabei wird der Fokus auf eine saubere Backend-Architektur, Datenbankbindung sowie sichere Authentifizierung gelegt.

In Modul 294 wird die Applikation um das dazugehörige Frontend ergänzt. Die REST-API bleibt unverändert die einzige Schnittstelle zur Datenhaltung; das Frontend setzt die beschriebenen Anwendungsfälle in einer Angular-Oberfläche um und macht sie für alle Benutzergruppen direkt im Browser zugänglich.

---

## Ziel des Projekts

Die Anwendung soll drei Benutzergruppen unterstützen:

- **Arbeitgeber**
- **Arbeitssuchende**
- **Administratoren**

Arbeitgeber sollen in der Lage sein, Stelleninserate zu erstellen, zu bearbeiten, anzuzeigen und zu löschen. Zusätzlich sollen sie alle eingegangenen Bewerbungen einsehen und diese akzeptieren oder ablehnen können.

Arbeitssuchende sollen Stelleninserate abrufen können, diese speichern und sich direkt über die Plattform auf offene Stellen bewerben können. Zudem sollen sie den Status ihrer Bewerbungen einsehen können.

Administratoren haben keine eigenen Inserate und Bewerbungen. Sie sind eine Moderationsrolle mit globaler Sicht: Sie können alle Benutzer und alle Bewerbungen einsehen, Bewerbungsstatus ändern sowie Benutzer und Inserate entfernen.

---

## Funktionale Anforderungen

Die Backend-Applikation stellt folgende Kernfunktionen bereit:

### 1. Benutzerverwaltung

Benutzer können sich registrieren und anmelden. Jeder Benutzer besitzt eine definierte Rolle:

- Arbeitgeber (**EMPLOYER**)
- Arbeitssuchender (**JOB\_SEEKER**)
- Administrator (**ADMIN**)

Diese Rollen bestimmen die verfügbaren Funktionen innerhalb der Anwendung.

Über die Registrierung können ausschliesslich die Rollen EMPLOYER und JOB\_SEEKER vergeben werden. Administratoren werden bewusst nur direkt in Keycloak angelegt und besitzen keinen Datensatz in der Datenbank.

---

## 2. Stelleninserate (Job Postings)

Arbeitgeber können Stelleninserate verwalten:

- Erstellen neuer Jobinserate (POST)
- Abrufen aller oder einzelner Stellen (GET)
- Bearbeiten bestehender Stellen (PUT/POST)
- Löschen von Stellen (DELETE)

Jede Stelle enthält Informationen wie Titel, Beschreibung, Standort und optional eine Gehaltsspanne.

## 3. Bewerbungen (Applications)

Arbeitssuchende können sich auf Stellen bewerben. Eine Bewerbung enthält:

- Referenz zur Stelle
- Referenz zum Benutzer
- Status der Bewerbung (pending, accepted, rejected)
- Zeitpunkt der Bewerbung

Arbeitgeber können eingegangene Bewerbungen einsehen und deren Status ändern.

## 4. Gespeicherte Stellen (Saved Jobs)

Arbeitssuchende können interessante Stellen speichern, um diese später schnell wiederzufinden. Diese Funktion dient als persönliche Merkliste.

# Technische Umsetzung

Das Backend wird mit **Spring Boot** realisiert und folgt einer REST-Architektur. Die Kommunikation erfolgt über HTTP-Requests.

## 1. Datenbank

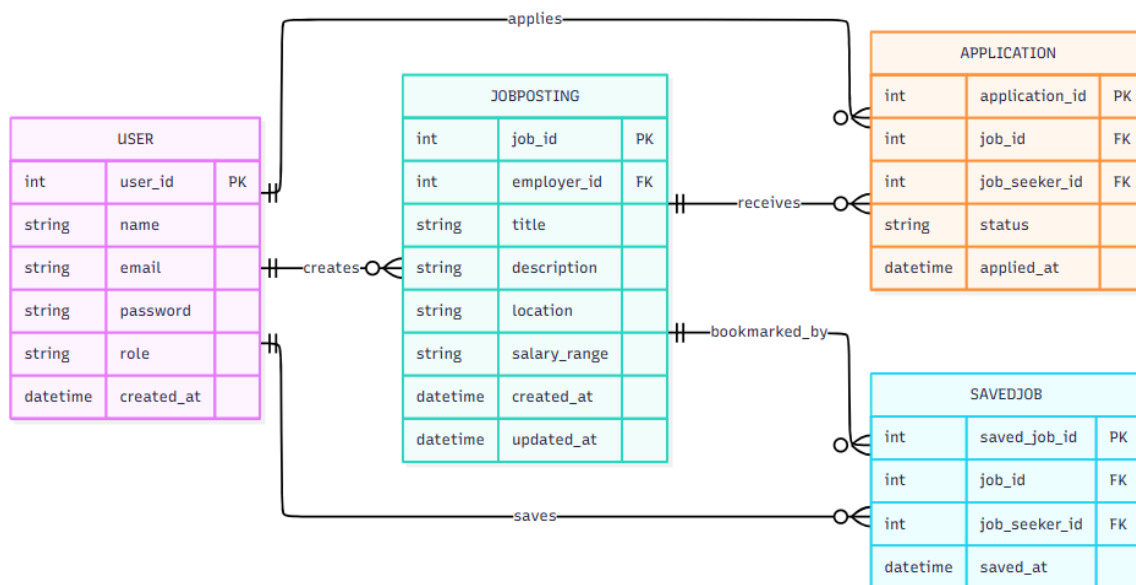
Als Datenbank wird **PostgreSQL** eingesetzt. Die Entitäten werden über **Spring Data JPA** abgebildet und verwaltet. Im `application.yml` müssen noch der Username und Passwort gesetzt werden.

Name der Datenbank: **JobMarketplace**

Folgende Hauptentitäten sind vorgesehen:

- User
- JobPosting
- Application
- SavedJob

ERM:

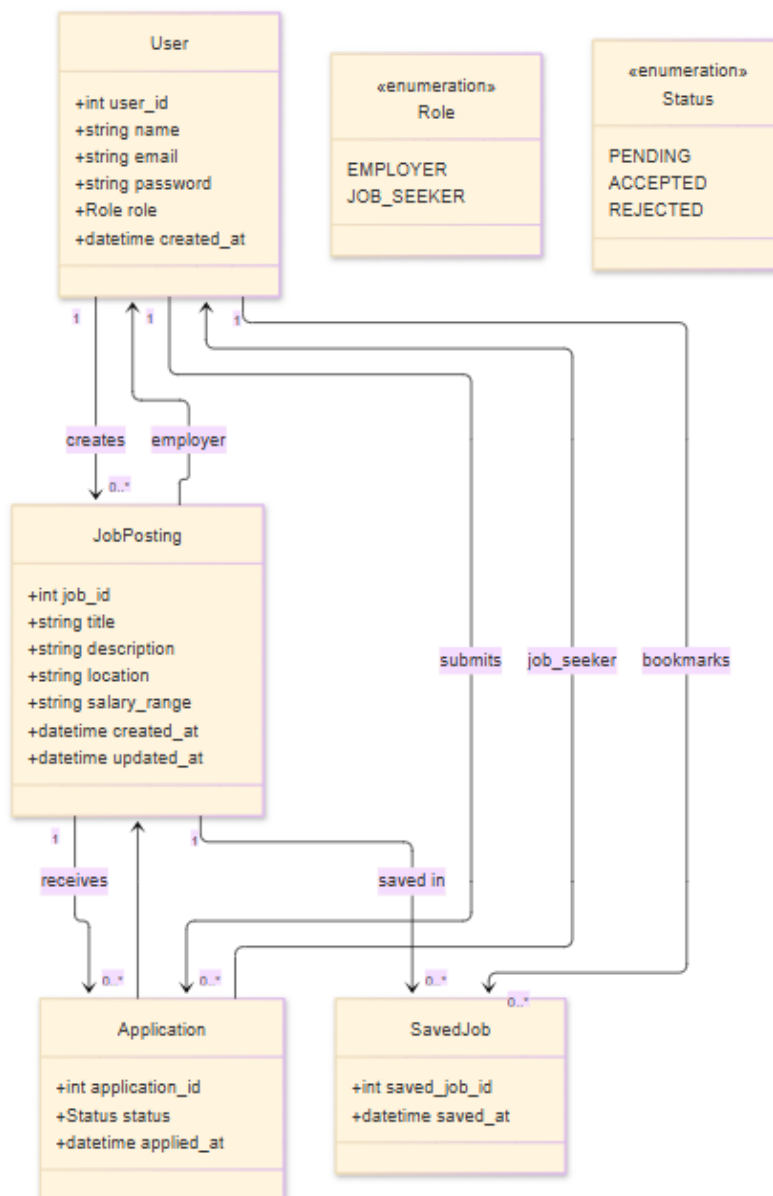


## 2. REST API Struktur

Die Applikation stellt mehrere REST-Controller bereit, unter anderem:

- User Controller (Authentifizierung und Benutzerverwaltung)
- Job Controller (CRUD für Stelleninserate)
- Application Controller (Bewerbungen verwalten)
- Saved Job Controller (gespeicherte Stellen verwalten)

UML:



---

### 3. Sicherheit

Die API wird durch **OAuth2 mit Keycloak** abgesichert. Dadurch wird sichergestellt, dass nur authentifizierte Benutzer Zugriff auf geschützte Ressourcen erhalten.

Zudem werden unterschiedliche Rollen (z. B. Arbeitgeber und Arbeitssuchende) verwendet, um den Zugriff auf Funktionen gezielt zu steuern.

#### Keycloak einrichten:

##### 1. Realm importieren

In der Keycloak Admin Console:

Create realm → Realm-Datei importieren → Realm erstellen

Der Realm-Name muss mit der `application.yaml` übereinstimmen.

##### 2. Client Secret kopieren

In Keycloak:

Clients → job-marketplace-api / job-marketplace-admin → Credentials → Client Secret kopieren

**Die Secrets müssen neu generiert werden!**

Danach das Secret in `application.yaml` eintragen:

keycloak:

```
server-url: http://localhost:8080
realm: job-marketplace
admin-client-id: job-marketplace-admin
admin-client-secret: HIER_SECRET_EINFÜGEN
login-client-id: job-marketplace-api
login-client-secret: HIER_SECRET_EINFÜGEN
```

spring:

```
security:
  oauth2:
    resourceserver:
      jwt:
        issuer-uri: http://localhost:8080/realms/job-marketplace
```

---

### 3. Client für das Frontend prüfen

Die Angular-Anwendung meldet sich über einen eigenen, öffentlichen Client an. Er ist im Realm-Export enthalten und muss nach dem Import folgende Einstellungen aufweisen:

| Einstellung                     | Wert                                                          |
|---------------------------------|---------------------------------------------------------------|
| Client ID                       | job-marketplace-frontend                                      |
| Client authentication           | off (public)                                                  |
| Standard flow                   | aktiviert                                                     |
| Direct access grants            | deaktiviert                                                   |
| Valid redirect URIs             | <a href="http://localhost:4200/">http://localhost:4200/</a> * |
| Valid post logout redirect URIs | <a href="http://localhost:4200/">http://localhost:4200</a>    |
| Web origins                     | <a href="http://localhost:4200/">http://localhost:4200</a>    |
| Full scope allowed              | aktiviert                                                     |

Ein öffentlicher Client besitzt kein Client Secret; die Absicherung erfolgt über PKCE. In der application.yml muss dieser Client deshalb nicht eingetragen werden — das Backend prüft eingehende Tokens nur gegen den Issuer des Realms, unabhängig davon, welcher Client sie ausgestellt hat.

Wichtig ist die Einstellung „Full scope allowed“: Ohne sie fehlen die Realm-Rollen im Access-Token, und im Frontend greifen weder Guards noch die rollenabhängige Anzeige.

### 4. Benutzer erstellen

Benutzer können entweder manuell in Keycloak erstellt werden oder über den Registrierungs-Endpoint:

**POST** /api/auth/register

Mögliche Rollen:

**ADMIN**  
**EMPLOYER**  
**JOB\_SEEKER**

**ADMIN** wird ausschliesslich manuell in Keycloak vergeben. Der Registrierungs-Endpoint weist die Rolle mit 400 Bad Request ab.



---

## 5. Token anfordern

Entweder direkt über Keycloak:

**POST** `http://localhost:8080/realms/job-marketplace/protocol/openid-connect/token`

oder über die Spring-Boot-Anwendung:

**POST** `/api/auth/login`

Das zurückgegebene **access\_token** wird für geschützte API-Endpunkte verwendet.

## 4. Anpassungen für die Frontend-Anbindung

Für die Anbindung des Angular-Frontends waren einige Ergänzungen am Backend nötig. Die fachliche Logik bleibt davon unberührt.

**Angemeldeter Benutzer.** Das Access-Token enthält als Subject nur die Keycloak-UUID des Benutzers, während alle benutzerbezogenen Endpoints die numerische `userId` aus der Datenbank erwarten. Der neue Endpoint `GET /api/users/me` löst diese Zuordnung auf. Er ist für alle angemeldeten Rollen freigegeben und liefert für Administratoren `404`, da diese bewusst keinen Datensatz in der Datenbank besitzen.

**Identität aus dem Token.** Die Create-Endpoints nahmen den handelnden Benutzer bisher als ID im Request-Body entgegen. Diese Felder sind entfallen; der Benutzer wird serverseitig aus dem Token bestimmt.

| Endpoint                                | bisher im Body           | neu           |
|-----------------------------------------|--------------------------|---------------|
| <code>POST /api/job-postings</code>     | <code>employerId</code>  | aus dem Token |
| <code>POST /api/job-applications</code> | <code>jobSeekerId</code> | aus dem Token |
| <code>POST /api/saved-jobs</code>       | <code>userId</code>      | aus dem Token |

`POST /api/job-postings` ist zudem auf die Rolle `EMPLOYER` beschränkt. Ein Inserat benötigt eine `employer_id` mit Fremdschlüssel auf die Tabelle `users`, und dort besitzt ein Administrator keinen Eintrag. Bearbeiten und Löschen von Inseraten bleiben für `ADMIN` als Moderationsfunktion erlaubt.

**CSRF-Schutz.** Der Schutz ist aktiviert und verwendet ein für JavaScript lesbares Cookie. Der erste Aufruf setzt ein `XSRF-TOKEN`-Cookie, das bei schreibenden Requests als Header `X-XSRF-TOKEN` zurückgesendet werden muss. Ausgenommen sind `POST /api/auth/register` und `POST`

---

/api/auth/login, damit Testbenutzer weiterhin direkt über die Swagger UI angelegt werden können.

**Statuscodes bei der Registrierung.** Die Registrierung meldet Fehler nicht mehr pauschal als Serverfehler, sondern über den Statuscode:

| Fall                                           | Status          |
|------------------------------------------------|-----------------|
| Registrierung erfolgreich                      | 201 Created     |
| Pflichtangabe fehlt oder Rolle ADMIN angefragt | 400 Bad Request |
| E-Mail-Adresse bereits vergeben                | 409 Conflict    |
| Keycloak nicht erreichbar                      | 502 Bad Gateway |

**Fehlerbehandlung in der Sicherheitskonfiguration.** Wirft ein Controller eine Exception, leitet Spring intern auf /error weiter. Dieser Forward durchläuft die Filterkette erneut. Ohne eine ausdrückliche Freigabe des Dispatcher-Typs ERROR fällt er auf die Regel „alle übrigen Anfragen müssen authentifiziert sein“ und liefert bei nicht angemeldeten Aufrufern 401 statt des tatsächlichen Statuscodes. Betroffen sind ausschliesslich öffentliche Endpoints, in der Praxis also die Registrierung.

## 5. Dokumentation

Die API wird mit **Swagger UI** dokumentiert, sodass alle Endpunkte einfach getestet und nachvollzogen werden können.

### Endpunkte:

#### AuthController:

| Methode | Endpoint                           | Beschreibung                                                                                                                                 |
|---------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| POST    | <a href="#">/api/auth/register</a> | Registriert einen neuen Benutzer und erstellt ihn in Keycloak sowie in der lokalen Datenbank. Zulässige Rollen sind EMPLOYER und JOB_SEEKER. |
| POST    | <a href="#">/api/auth/login</a>    | Meldet einen Benutzer über Keycloak an und gibt ein Access Token zurück.                                                                     |

---

## UserController:

| Methode | Endpoint                                      | Beschreibung                                                                                     |
|---------|-----------------------------------------------|--------------------------------------------------------------------------------------------------|
| GET     | <code>/api/users/me</code>                    | Gibt den angemeldeten Benutzer samt seiner lokalen ID zurück.                                    |
| GET     | <code>/api/users</code>                       | Gibt eine Liste aller Benutzer zurück.                                                           |
| GET     | <code>/api/users/{userId}</code>              | Gibt einen einzelnen Benutzer anhand seiner ID zurück.                                           |
| PUT     | <code>/api/users/{userId}</code>              | Aktualisiert einen bestehenden Benutzer.                                                         |
| DELETE  | <code>/api/users/{userId}</code>              | Löscht einen bestehenden Benutzer.                                                               |
| GET     | <code>/api/users/{userId}/job-postings</code> | Gibt alle Stelleninserate zurück, die von einem bestimmten Benutzer/Arbeitgeber erstellt wurden. |

## JobPostingController:

| Methode | Endpoint                               | Beschreibung                                               |
|---------|----------------------------------------|------------------------------------------------------------|
| GET     | <code>/api/job-postings</code>         | Gibt eine Liste aller Stelleninserate zurück.              |
| POST    | <code>/api/job-postings</code>         | Erstellt ein neues Stelleninserat.                         |
| GET     | <code>/api/job-postings/{jobId}</code> | Gibt ein einzelnes Stelleninserat anhand seiner ID zurück. |

---

|        |                                        |                                              |
|--------|----------------------------------------|----------------------------------------------|
| PUT    | <code>/api/job-postings/{jobId}</code> | Aktualisiert ein bestehendes Stelleninserat. |
| DELETE | <code>/api/job-postings/{jobId}</code> | Löscht ein bestehendes Stelleninserat.       |

### JobApplicationController:

| Methode | Endpoint                                                  | Beschreibung                                                                        |
|---------|-----------------------------------------------------------|-------------------------------------------------------------------------------------|
| GET     | <code>/api/job-applications</code>                        | Gibt eine Liste aller Bewerbungen zurück.                                           |
| POST    | <code>/api/job-applications</code>                        | Erstellt eine neue Bewerbung für ein Stelleninserat.                                |
| GET     | <code>/api/job-applications/{applicationId}</code>        | Gibt eine einzelne Bewerbung anhand ihrer ID zurück.                                |
| PATCH   | <code>/api/job-applications/{applicationId}/status</code> | Aktualisiert den Status einer Bewerbung.                                            |
| DELETE  | <code>/api/job-applications/{applicationId}</code>        | Löscht eine bestehende Bewerbung.                                                   |
| GET     | <code>/api/users/{userId}/job-applications</code>         | Gibt alle Bewerbungen zurück, die von einem bestimmten Benutzer eingereicht wurden. |

---

|     |                                                         |                                                                 |
|-----|---------------------------------------------------------|-----------------------------------------------------------------|
| GET | <code>/api/job-postings/{jobId}/job-applications</code> | Gibt alle Bewerbungen für ein bestimmtes Stelleninserat zurück. |
|-----|---------------------------------------------------------|-----------------------------------------------------------------|

#### **SavedJobController:**

| Methode | Endpoint                                    | Beschreibung                                                           |
|---------|---------------------------------------------|------------------------------------------------------------------------|
| GET     | <code>/api/saved-jobs</code>                | Gibt eine Liste aller gespeicherten Stellen zurück.                    |
| POST    | <code>/api/saved-jobs</code>                | Speichert ein Stelleninserat als Merkliste-Eintrag für einen Benutzer. |
| GET     | <code>/api/saved-jobs/{savedJobId}</code>   | Gibt eine einzelne gespeicherte Stelle anhand ihrer ID zurück.         |
| DELETE  | <code>/api/saved-jobs/{savedJobId}</code>   | Entfernt eine gespeicherte Stelle.                                     |
| GET     | <code>/api/users/{userId}/saved-jobs</code> | Gibt alle gespeicherten Stellen eines bestimmten Benutzers zurück.     |

## **5. Versionsverwaltung**

Das gesamte Projekt wird über Git versioniert und gepflegt. Dadurch wird eine saubere Entwicklungsstruktur sowie Nachvollziehbarkeit der Änderungen gewährleistet.

- Nur Backend: <https://github.com/bawan-code/m295>
- Backend & Frontend: <https://github.com/bawan-code/m294>

# Frontend

## Ausgangslage und Ziel

Das in Modul 295 erstellte Backend stellt die vollständige REST-API der Job-Marketplace-Applikation bereit, war bisher aber nur über die Swagger UI bedienbar. In diesem Modul wird das dazugehörige Frontend mit Angular entwickelt.

Ziel ist eine Oberfläche, welche die bereits beschriebenen Anwendungsfälle abbildet und dabei die Rollen des Backends konsequent berücksichtigt. Arbeitssuchende sollen Stellen durchsuchen, sich bewerben und ihre Bewerbungen verfolgen können. Arbeitgeber sollen ihre Inserate verwalten und eingegangene Bewerbungen bearbeiten können. Administratoren erhalten eine globale Sicht auf Benutzer und Bewerbungen.

Die Sicherheitsvorgaben des Backends werden dabei vollständig übernommen: Anmeldung über Keycloak mit OAuth2, Mitführen des Access-Tokens bei jedem Request, XSRF-Token bei schreibenden Zugriffen und Beachtung der CORS-Einstellungen.

## 1. Ansichten und Wireframes

Vor der Umsetzung wurden die benötigten Ansichten festgelegt und als Wireframes skizziert. Die folgende Übersicht zeigt, welche Ansicht welchem Zweck dient und für wen sie zugänglich ist.

**1. Dashboard** – Einstieg, Anmeldung, rollenabhängige Zugriffe (sichtbar für alle)

Ausgeloggt:

Bitte melden Sie sich an, um die Anwendung zu nutzen.

[Anmelden]

Angemeldet:

Willkommen, Max Muster (max@muster.ch)

|                     |                   |            |
|---------------------|-------------------|------------|
| Stellen durchsuchen | Meine Bewerbungen | JOB_SEEKER |
| Meine Inserate      | Neue Stelle       | EMPLOYER   |
| Benutzerverwaltung  |                   | ADMIN      |

## 2. Stellenliste - Übersicht aller ausgeschriebenen Stellen (alle, auch ohne Anmeldung)

| Stellen                             |        |          |             | [+ Neue Stelle erfassen]<br>^ nur EMPLOYER |
|-------------------------------------|--------|----------|-------------|--------------------------------------------|
| Titel                               | Ort    | Gehalt   | Arbeitgeber | Aktionen                                   |
| Senior Dev                          | Zürich | 90k-110k | Muster AG   | [Details]<br>[Merken]                      |
|                                     |        |          |             |                                            |
| Einträge pro Seite [10 v] 1-2 von 2 |        |          |             | < < > >                                    |

## 3. Stellendetail - Einzelne Stelle mit rollenabhängigen Aktionen (alle, auch ohne Anmeldung)

| [<- Zurück]                      Stellendetail                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Senior Java Entwickler<br>Zürich - 90'000 - 110'000 CHF<br>Arbeitgeber: Muster AG<br>erstellt am 12.03.2026<br><br>Beschreibung:<br>Wir suchen eine erfahrene Entwicklerin oder einen.... |
| AKTIONSLEISTE - rollenabhängig eingeblendet:<br>anonym: (keine Buttons)<br>JOB_SEEKER: [Jetzt bewerben]    [Job merken]<br>EMPLOYER: [Bearbeiten] [Löschen] [Bewerbungen ansehen]         |

---

#### 4. Kein Zugriff - Hinweis bei fehlender Berechtigung (alle)

!!

Sie haben keinen Zugriff auf diese Seite.

[ Anmelden ]

^ nur wenn nicht angemeldet

#### 5. Meine Bewerbungen – Eigene Bewerbungen samt Status (Arbeitssuchende)

| Meine Bewerbungen |             |                         |          |                |
|-------------------|-------------|-------------------------|----------|----------------|
| Stelle            | Arbeitgeber | Status                  | Beworben | Aktion         |
|                   |             | (Offen)<br>(Angenommen) |          | [Zurücknehmen] |
| [[Paginator]]     |             |                         |          |                |



## 6. Gemerkte Jobs – Persönliche Merkliste (Arbeitssuchende)

| Gemerkte Jobs |     |        |             |            |                          |
|---------------|-----|--------|-------------|------------|--------------------------|
| Stelle        | Ort | Gehalt | Arbeitgeber | Gemerkt am | Aktionen                 |
|               |     |        |             |            | [Zum Job]<br>[Entfernen] |
| [[Paginator]] |     |        |             |            |                          |

## 7. Meine Inserate – Eigene Stelleninserate verwalten (Arbeitgeber)

| Meine Inserate |     |        |          | [+Neue Stelle erfassen]                    |
|----------------|-----|--------|----------|--------------------------------------------|
| Stelle         | Ort | Gehalt | Erstellt | Aktionen                                   |
|                |     |        |          | [Bearbeiten]<br>[Bewerbungen]<br>[Löschen] |
| [[Paginator]]  |     |        |          |                                            |

## 8. Inserat erfassen / bearbeiten – Formular für ein Stelleninserat (Arbeitgeber)

| [<-Zurück]                      Stelle bearbeiten                                       |          |
|-----------------------------------------------------------------------------------------|----------|
| Titel *<br>[Senior Java Entwickler_____]                                                | 23/100   |
| Beschreibung *<br>[Wir suchen eine erfahrene Entwicklerin...____]<br>[_____]            | 412/2000 |
| Ort *<br>[_____]<br>(!) Ort ist erforderlich!      <- Mat-Error                         | 0/100    |
| Gehaltsspanne<br>[Ungefähr 90k_____]<br>(!) Format z.B.: 90000-110000      <- Mat-Error |          |
| [Speichern]    <- Deaktiviert, solange ungültig                                         |          |

## 9. Bewerbungen auf ein Inserat – Eingegangene Bewerbungen annehmen oder ablehnen (Arbeitgeber)

| [<- Zurück]                      Bewerbungen: Senior Java Entwickler |        |                                        |    |                          |
|----------------------------------------------------------------------|--------|----------------------------------------|----|--------------------------|
| Bewerber                                                             | E-Mail | Status                                 | Am | Aktionen                 |
|                                                                      |        | (Offen)<br>(Abgelehnt)<br>(Akzeptiert) |    | [Annehmen]<br>[Ablehnen] |
| [[Paginator]]                                                        |        |                                        |    |                          |

## 10. Mein Profil – Name und E-Mail-Adresse ändern (Arbeitgeber, Arbeitssuchende)

|                                                                      |  |               |  |
|----------------------------------------------------------------------|--|---------------|--|
| [<-Zurück]                                                           |  | Mein Profil   |  |
| Name *                                                               |  |               |  |
| [Max Musterman_____]                                                 |  | 23/100        |  |
| E-Mail *                                                             |  |               |  |
| [max.muster@_____]                                                   |  | 12/100        |  |
| ( ! ) Bitte eine gültige E-Mail-Adresse eingeben <- Validators.email |  |               |  |
| Rolle                                                                |  |               |  |
| [JOB_SEEKER_____]                                                    |  | (deaktiviert) |  |
| [Speichern]                                                          |  |               |  |

## 11. Benutzerverwaltung – Alle Benutzer einsehen und Entfernen (Administratoren)

| Benutzerverwaltung |        |                            |          |           |
|--------------------|--------|----------------------------|----------|-----------|
| Bewerber           | E-Mail | Rolle                      | Erstellt | Aktionen  |
|                    |        | (JOB_SEEKER)<br>(EMPLOYER) |          | [Löschen] |
|                    |        | [[Paginator]]              |          |           |

## 12. Wiederverwendete Bausteine

Bestätigungsdialog - von 5 Seiten genutzt:

|                                              |
|----------------------------------------------|
| Sind Sie sicher?                             |
| Möchten sie diesen Eintrag wirklich löschen? |
| [Nein]   [Ja]                                |

Status-Badge — drei Zustände:

|              |           |
|--------------|-----------|
| (Offen)      | gelb/grau |
| (Angenommen) | grün      |
| (Abgelehnt)  | rot       |

## 2. Komponentenübersicht

Die Anwendung ist wie das Demoprojekt in Pages und Einzelkomponenten unterteilt. Pages werden über das Routing angesteuert und entsprechen jeweils einer vollständigen Ansicht. Einzelkomponenten werden innerhalb von Pages wiederverwendet.

Insgesamt sind 18 Komponenten entstanden: 13 Pages und 5 Einzelkomponenten.

### Pages

| Komponente                             | Route                          | Rollen               | Aufgabe                                         |
|----------------------------------------|--------------------------------|----------------------|-------------------------------------------------|
| <b>DashboardComponent</b>              | /, /dashboard                  | alle                 | Anmeldung und rollenabhängige Schnellzugriffe   |
| <b>JobListComponent</b>                | /jobs                          | alle                 | Tabelle aller Stelleninserate mit Paginierung   |
| <b>JobDetailComponent</b>              | /jobs/:id                      | alle                 | Einzelnes Inserat mit rollenabhängigen Aktionen |
| <b>RegisterComponent</b>               | /register                      | nicht angemeldet     | Registrierungsformular                          |
| <b>NoAccessComponent</b>               | /noaccess                      | alle                 | Hinweis bei fehlender Berechtigung              |
| <b>MyApplicationsComponent</b>         | /my-applications               | JOB_SEEKER           | Eigene Bewerbungen, Zurückziehen                |
| <b>SavedJobsComponent</b>              | /saved-jobs                    | JOB_SEEKER           | Merkliste, Entfernen                            |
| <b>MyJobPostingsComponent</b>          | /my-job-postings               | EMPLOYER             | Eigene Inserate verwalten                       |
| <b>JobPostingDetailComponent</b>       | /job-posting, /job-posting/:id | EMPLOYER             | Inserat erfassen und bearbeiten                 |
| <b>JobPostingApplicationsComponent</b> | /job-postings/:id/applications | EMPLOYER             | Bewerbungen annehmen oder ablehnen              |
| <b>ProfileComponent</b>                | /profile                       | EMPLOYER, JOB_SEEKER | Eigene Stammdaten ändern                        |
| <b>UserListComponent</b>               | /users                         | ADMIN                | Benutzerverwaltung                              |
| <b>AllApplicationsComponent</b>        | /all-applications              | ADMIN                | Globale Sicht auf alle Bewerbungen              |

### Einzelkomponenten

| Komponente                | Verwendet in            | Aufgabe                                       |
|---------------------------|-------------------------|-----------------------------------------------|
| <b>AppHeaderComponent</b> | Grundgerüst             | Kopfzeile mit Anwendungs- und Seitentitel     |
| <b>AppLoginComponent</b>  | Dashboard, Kein Zugriff | Schaltflächen für Anmeldung und Registrierung |

|                               |                                                                        |                                            |
|-------------------------------|------------------------------------------------------------------------|--------------------------------------------|
| <b>ConfirmDialogComponent</b> | sieben Seiten                                                          | Rückfrage vor Löschen und Statusänderungen |
| <b>StatusBadgeComponent</b>   | Meine Bewerbungen,<br>Bewerbungen auf ein Inserat,<br>Alle Bewerbungen | Farbige Darstellung des Bewerbungsstatus   |
| <b>BaseComponent</b>          | alle Listen- und Detailseiten                                          | Basisklasse mit den Rückmeldungstexten     |

Ergänzend zu den Komponenten wurden erstellt:

- drei Direktiven: `IsInRoleDirective` und `IsInRolesDirective` für die rollenabhängige Sichtbarkeit, `AutofocusDirective` für den Eingabefokus in Formularen
- ein Route-Guard `appCanActivate` für die Rollenprüfung beim Seitenaufruf
- ein Interceptor `HttpXSRFInterceptor`, der das XSRF-Token an schreibende Requests anhängt

### 3. Routing

Das Routing ist in `app.routes.ts` hinterlegt. Geschützte Routen erhalten den Guard `appCanActivate` und die erforderlichen Rollen als Routendaten.

| Route                          | Komponente                      | Guard                       | Erforderliche Rollen |
|--------------------------------|---------------------------------|-----------------------------|----------------------|
| /                              | DashboardComponent              | —                           | —                    |
| /dashboard                     | DashboardComponent              | —                           | —                    |
| /jobs                          | JobListComponent                | —                           | —                    |
| /jobs/:id                      | JobDetailComponent              | —                           | —                    |
| /register                      | RegisterComponent               | —                           | —                    |
| /noaccess                      | NoAccessComponent               | —                           | —                    |
| /my-job-postings               | MyJobPostingsComponent          | <code>appCanActivate</code> | EMPLOYER             |
| /job-posting                   | JobPostingDetailComponent       | <code>appCanActivate</code> | EMPLOYER             |
| /job-posting/:id               | JobPostingDetailComponent       | <code>appCanActivate</code> | EMPLOYER             |
| /job-postings/:id/applications | JobPostingApplicationsComponent | <code>appCanActivate</code> | EMPLOYER             |
| /my-applications               | MyApplicationsComponent         | <code>appCanActivate</code> | JOB_SEEKER           |
| /saved-jobs                    | SavedJobsComponent              | <code>appCanActivate</code> | JOB_SEEKER           |
| /profile                       | ProfileComponent                | <code>appCanActivate</code> | EMPLOYER, JOB_SEEKER |
| /users                         | UserListComponent               | <code>appCanActivate</code> | ADMIN                |
| /all-applications              | AllApplicationsComponent        | <code>appCanActivate</code> | ADMIN                |

Der Guard prüft zuerst, ob ein gültiges Access-Token vorliegt, und anschliessend, ob der Benutzer mindestens eine der geforderten Rollen besitzt. Trifft eines von beidem nicht zu, wird auf `/noaccess` umgeleitet. Das gilt auch dann, wenn eine geschützte Adresse direkt in die Adresszeile eingegeben wird. Unbekannte Adressen werden auf das Dashboard umgeleitet.

Die Stellenliste und das Stellendetail sind bewusst ohne Guard erreichbar, weil das Backend die entsprechenden GET-Endpoints ebenfalls öffentlich freigibt. Die Registrierung ist ebenfalls ungeschützt, da sich ein neuer Benutzer naturgemäss noch nicht anmelden kann.

## 4. Services und Backend-Anbindung

Für jede Ressource des Backends existiert ein eigener Service. Alle Services sind anwendungsweit bereitgestellt, verwenden den HttpClient und bilden ihre Basisadresse aus der Umgebungsconfiguration und einem festen Pfadbestandteil.

| Service                      | Angebundene Endpoints                                                                            | Implementierte Methoden                                            |
|------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>JobPostingService</b>     | /api/job-postings, /api/users/{id}/job-postings                                                  | getList, getOne, save, update, delete, getListByUser               |
| <b>JobApplicationService</b> | /api/job-applications, /api/users/{id}/job-applications, /api/job-postings/{id}/job-applications | getList, getOne, save, update, delete, getListByUser, getListByJob |
| <b>SavedJobService</b>       | /api/saved-jobs, /api/users/{id}/saved-jobs                                                      | getList, getOne, save, delete, getListByUser                       |
| <b>UserService</b>           | /api/users                                                                                       | getList, getOne, getMe, update, delete                             |
| <b>RegistrationService</b>   | /api/auth/register                                                                               | register                                                           |

JobPostingService ist der Service mit dem vollständigen CRUD-Umfang: Er deckt Anlegen, Lesen, Ändern und Löschen vollständig ab.

Ergänzend gibt es drei Services ohne direkten Bezug zu einer Ressource:

- AppAuthService kapselt die Anmeldung über Keycloak und stellt Benutzername und Rollen bereit.
- CurrentUserService lädt über GET /api/users/me die numerische Benutzer-ID und hält sie für alle benutzerbezogenen Seiten vor.
- HeaderService übermittelt den Seitentitel an die Kopfzeile.

**Mapping zwischen JSON und Klassen.** Zu jedem Response-DTO des Backends existiert im Verzeichnis dataaccess eine Modellklasse mit denselben Feldnamen: JobPosting, JobApplication, SavedJob, User und RegisterRequest. Für die Aufzählungstypen Role und Status werden Typdefinitionen verwendet. Da die Response-DTOs des Backends flach aufgebaut sind und verknüpfte Angaben wie employerName oder jobTitle bereits mitliefern, sind im Frontend keine verschachtelten Objekte und keine Nachladelogik nötig.

## 5. Rollen und Berechtigungen

Der Zugriffsschutz ist zweistufig aufgebaut. Ganze Seiten werden über den Route-Guard gesperrt, einzelne Teile einer Seite über Direktiven ein- oder ausgeblendet.

**Funktion / Ansicht**      **ADMIN**   **EMPLOYER**   **JOB\_SEEKER**   **anonym**   **Mechanismus**

|                                               |      |      |      |      |                      |
|-----------------------------------------------|------|------|------|------|----------------------|
| <b>Stellen durchsuchen</b>                    | ja   | ja   | ja   | ja   | öffentlich           |
| <b>Stellendetail ansehen</b>                  | ja   | ja   | ja   | ja   | öffentlich           |
| <b>Auf eine Stelle bewerben</b>               | nein | nein | ja   | nein | Direktive            |
| <b>Stelle merken</b>                          | nein | nein | ja   | nein | Direktive            |
| <b>Meine Bewerbungen</b>                      | nein | nein | ja   | nein | Guard                |
| <b>Gemerkte Jobs</b>                          | nein | nein | ja   | nein | Guard                |
| <b>Inserat erfassen</b>                       | nein | ja   | nein | nein | Guard +<br>Direktive |
| <b>Inserat bearbeiten oder löschen</b>        | nein | ja   | nein | nein | Guard +<br>Direktive |
| <b>Bewerbungen auf ein Inserat bearbeiten</b> | nein | ja   | nein | nein | Guard                |
| <b>Eigenes Profil ändern</b>                  | nein | ja   | ja   | nein | Guard                |
| <b>Benutzerverwaltung</b>                     | ja   | nein | nein | nein | Guard                |
| <b>Alle Bewerbungen</b>                       | ja   | nein | nein | nein | Guard                |

Administratoren stehen bewusst keine benutzerbezogenen Seiten offen. Da sie nur in Keycloak existieren und keinen Datensatz in der Datenbank besitzen, gibt es für sie weder eigene Inserate noch eigene Bewerbungen. Ihnen werden stattdessen die globalen Ansichten angeboten, die ohne Benutzer-ID auskommen.

**Rollenabhängige Teile einer Seite.** Am deutlichsten zeigt sich dies auf der Detailseite eines Stelleninserats. Es handelt sich um eine einzige, öffentlich erreichbare Adresse, deren Aktionsleiste je nach angemeldeter Rolle unterschiedlich ausfällt:

| Rolle            | Sichtbare Aktionen                             |
|------------------|------------------------------------------------|
| nicht angemeldet | keine Aktionen, nur die Angaben zur Stelle     |
| Arbeitssuchender | „Jetzt bewerben“, „Job merken“                 |
| Arbeitgeber      | „Bearbeiten“, „Bewerbungen ansehen“, „Löschen“ |

Weitere Beispiele sind die Navigationsleiste, deren Einträge je Rolle unterschiedlich sind, die Schaltfläche „Neue Stelle erfassen“ in der Stellenliste und die Schnellzugriffe auf dem Dashboard.

## 6. Sicherheit im Frontend

**Anmeldung.** Die Anmeldung erfolgt über Keycloak im Authorization Code Flow mit PKCE. Dafür wird die Bibliothek angular-oauth2-oidc verwendet. Der Benutzer wird zur Anmeldemaske von Keycloak weitergeleitet und nach erfolgreicher Anmeldung zurück zur Anwendung geführt. Die Abmeldung erfolgt ebenfalls über Keycloak und verwirft das Token.

**Access-Token.** Das Token wird in der Sitzung des Browsers abgelegt und automatisch an alle Requests an das Backend angehängt.

**Rollen.** Die Rollen werden aus dem Claim `realm_access.roles` des Access-Tokens gelesen. Es handelt sich um Realm-Rollen, die in Grossbuchstaben und ohne Präfix vorliegen. Sie werden

---

über ein BehaviorSubject bereitgestellt, damit Guard und Direktiven die Rollen auch dann erhalten, wenn das Token erst nach ihrer Initialisierung eintrifft.

**XSRF-Token.** Ein Interceptor liest das vom Backend gesetzte XSRF-TOKEN-Cookie und hängt es bei schreibenden Requests als Header X-XSRF-TOKEN an. Requests an Keycloak sind davon ausgenommen.

**CORS.** Das Backend gibt ausschliesslich http://localhost:4200 frei. Werden Frontend oder Backend auf anderen Ports betrieben, müssen sowohl die CORS-Einstellung im Backend als auch die Redirect-URLs und Web-Origins des Keycloak-Clients angepasst werden.

## 7. Validierung der Eingabedaten

Alle Formulare sind als reaktive Formulare umgesetzt. Die Regeln sind im TypeScript-Code hinterlegt, Verstösse werden unterhalb des betroffenen Feldes im Klartext angezeigt, und die Schaltfläche zum Speichern bleibt gesperrt, solange das Formular ungültig ist. Bei längeren begrenzten Feldern zeigt ein Zähler die verbrauchten Zeichen an.

### Inserat erfassen und bearbeiten

| Feld          | Regel                               |
|---------------|-------------------------------------|
| Titel         | Pflichtfeld, 3 bis 100 Zeichen      |
| Beschreibung  | Pflichtfeld, 10 bis 2000 Zeichen    |
| Ort           | Pflichtfeld, höchstens 100 Zeichen  |
| Gehaltsspanne | optional, Muster 90000 - 110000 CHF |

### Profil

| Feld   | Regel                                                      |
|--------|------------------------------------------------------------|
| Name   | Pflichtfeld, höchstens 100 Zeichen                         |
| E-Mail | Pflichtfeld, gültige E-Mail-Adresse, höchstens 150 Zeichen |
| Rolle  | nur zur Anzeige, nicht änderbar                            |

### Registrierung

| Feld                | Regel                                                        |
|---------------------|--------------------------------------------------------------|
| Name                | Pflichtfeld, 2 bis 100 Zeichen                               |
| E-Mail              | Pflichtfeld, gültige E-Mail-Adresse, höchstens 150 Zeichen   |
| Passwort            | Pflichtfeld, 8 bis 64 Zeichen                                |
| Passwort bestätigen | Pflichtfeld, muss mit dem Passwort übereinstimmen            |
| Rolle               | Pflichtfeld, Auswahl zwischen Arbeitgeber und Arbeitssuchend |

Neben der Prüfung einzelner Felder kommt bei der Registrierung eine Prüfung über das gesamte Formular zum Einsatz: Ein eigens geschriebener Validator vergleicht die beiden Passwortfelder und meldet eine Abweichung, sobald beide Felder ausgefüllt sind.

Die Passwortregel ist bewusst im Frontend verankert. Der Realm in Keycloak besitzt keine Passwortrichtlinie, weshalb ohne diese Prüfung jedes beliebige Passwort zulässig wäre.



---

## 8. Einrichtung und Start

Voraussetzungen:

- Node 24
- PostgreSQL mit der Datenbank [JobMarketplace](#)
- Keycloak mit dem importierten Realm [job-marketplace](#)
- laufendes Backend auf [Port 9090](#)

Vorgehen:

1. In das Verzeichnis frontend wechseln.
2. Abhängigkeiten installieren:  
`npm install`
3. Anwendung starten:  
`npm start`
4. Die Anwendung ist anschliessend unter <http://localhost:4200> erreichbar.

Weitere Befehle:

| Befehl                     | Zweck                      |
|----------------------------|----------------------------|
| <code>npm test</code>      | Unit-Tests ausführen       |
| <code>npm run lint</code>  | Codestil prüfen            |
| <code>npm run build</code> | Produktiven Build erzeugen |

## 9. Kurzanleitung

Die Anleitung führt einmal durch die Anwendung: zuerst das Anlegen eines Kontos, danach die Anmeldung und anschliessend die Funktionen der drei Benutzergruppen. Die Anmeldung ist für alle Rollen gleich und wird deshalb nur einmal beschrieben. Die Abschnitte ab „Arbeitssuchender“ setzen jeweils voraus, dass ein Benutzer der genannten Rolle angemeldet ist.

### 1. Registrierung

1. Die Anwendung unter `http://localhost:4200` öffnen. Ohne Anmeldung zeigt das Dashboard die Schaltflächen „Anmelden“ und „Registrieren“.
2. „Registrieren“ wählen.
3. Name, E-Mail-Adresse und Passwort erfassen und die gewünschte Rolle auswählen. Zur Wahl stehen „Arbeitssuchend“ und „Arbeitgeber“.
4. Solange Angaben fehlen oder ungültig sind, bleibt „Registrieren“ gesperrt und die betroffenen Felder zeigen einen Hinweis. Stimmen die beiden Passwortfelder nicht überein, erscheint eine Meldung unterhalb der Eingabe.
5. Ist die E-Mail-Adresse bereits vergeben, meldet die Anwendung dies im Klartext. Die Eingaben bleiben erhalten und können korrigiert werden.
6. Nach erfolgreicher Registrierung ersetzt eine Bestätigung das Formular. Über die Schaltfläche „Jetzt anmelden“ geht es direkt weiter zur Anmeldung.

Bitte melden Sie sich an, um die Anwendung zu nutzen.

→ Anmelden

+ Registrieren

Name\*

Bawan

5/100

E-Mail\*

bawan@bawan.ch

14/150

Passwort\*

.....

mindestens 8 Zeichen

Passwort bestätigen\*

.....

Ich registriere mich als\*

Arbeitssuchend



Registrieren



## Konto erstellt

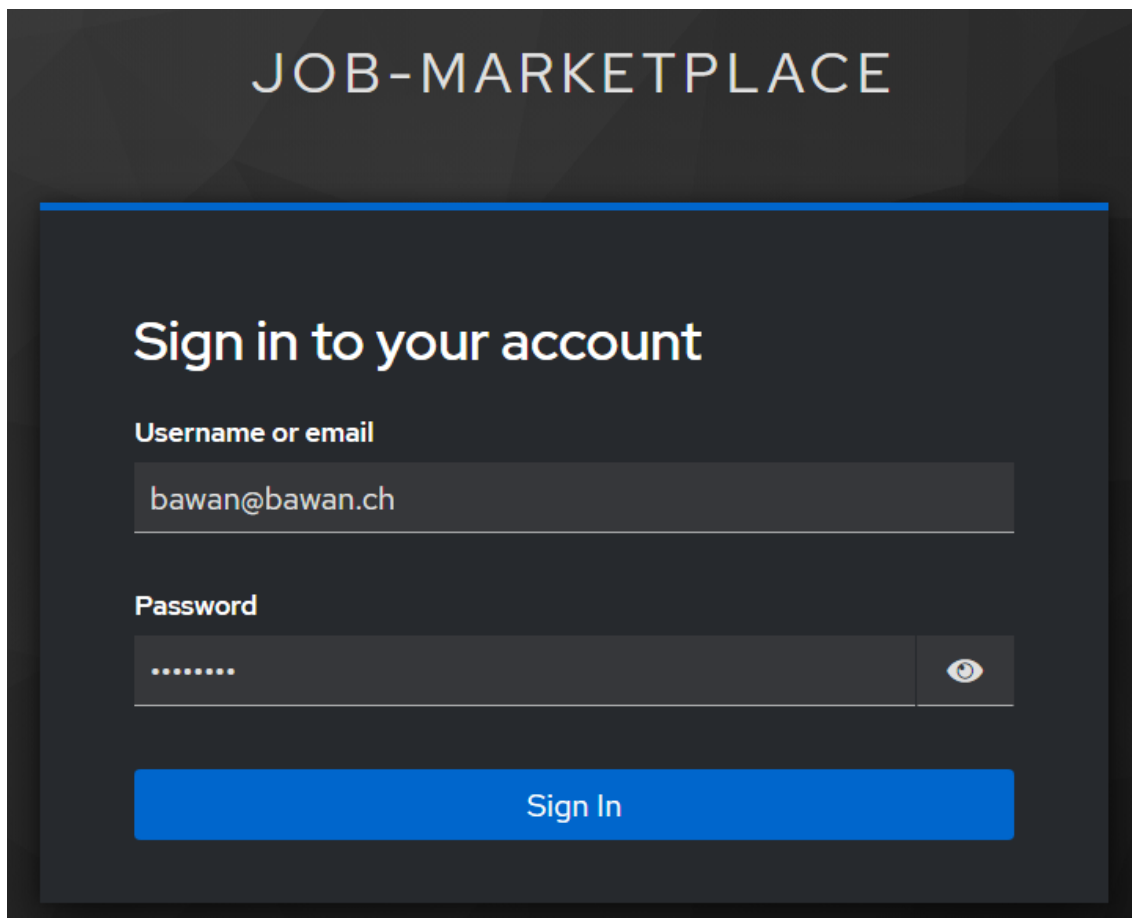
Sie können sich jetzt mit Ihrer E-Mail-Adresse und Ihrem Passwort anmelden.

→ Jetzt anmelden

## 2. Anmeldung

Die Anmeldung läuft für alle Benutzergruppen identisch ab.

1. Auf dem Dashboard „Anmelden“ wählen. Derselbe Einstieg erscheint auch auf der Seite „Kein Zugriff“, wenn ein geschützter Bereich ohne Anmeldung aufgerufen wurde.
2. Die Anwendung leitet auf die Anmeldemaske von Keycloak weiter.
3. Dort E-Mail-Adresse und Passwort eingeben.
4. Nach erfolgreicher Anmeldung kehrt die Anwendung zum Dashboard zurück. Die Kopfzeile zeigt den angemeldeten Benutzer, und die Navigationsleiste enthält nur noch die Einträge, die der jeweiligen Rolle offenstehen.
5. Abgemeldet wird über „Abmelden“ am unteren Ende der Navigationsleiste.



**JOB-MARKETPLACE**

**Sign in to your account**

**Username or email**

bawan@bawan.ch

**Password**

.....

**Sign In**

Willkommen Bawan - (bawan@bawan.ch)!

 Stellen durchsuchen

 Meine Bewerbungen

 Gemerkte Jobs

### 3. Arbeitssuchender

1. Über „Stellen“ die Liste der ausgeschriebenen Stellen öffnen.
2. Bei einer Stelle „Details“ wählen. Auf der Detailseite stehen die Aktionen „Jetzt bewerben“ und „Job merken“ zur Verfügung.
3. „Jetzt bewerben“ anklicken. Eine Rückmeldung bestätigt die eingereichte Bewerbung.
4. „Job merken“ wählen. Die Stelle erscheint anschliessend unter „Gemerkte Jobs“ und lässt sich dort nach Rückfrage wieder entfernen.
5. Unter „Meine Bewerbungen“ den Status verfolgen. Er wird farbig als „Offen“, „Angenommen“ oder „Abgelehnt“ dargestellt.
6. Über „Zurückziehen“ lässt sich eine Bewerbung nach Rückfrage entfernen.
7. Unter „Profil“ können Name und E-Mail-Adresse geändert werden. Die Rolle wird angezeigt, ist aber nicht änderbar.

#### Stellen

| Titel                         | Ort        | Gehalt              | Arbeitgeber | Aktionen                                       |
|-------------------------------|------------|---------------------|-------------|------------------------------------------------|
| Frontend Entwicklerin Angular | Bern       | 85000 - 100000 CHF  | employer    | <a href="#">Details</a> <a href="#">Merken</a> |
| Data Analyst                  | Zürich     | 88000 - 105000 CHF  | employer    | <a href="#">Details</a> <a href="#">Merken</a> |
| Systemadministrator           | Luzern     | 75000 - 90000 CHF   | employer    | <a href="#">Details</a> <a href="#">Merken</a> |
| Senior Java Entwickler        | Zürich     | 95000 - 115000 CHF  | employer    | <a href="#">Details</a> <a href="#">Merken</a> |
| DevOps Engineer               | Basel      | 100000 - 120000 CHF | employer    | <a href="#">Details</a> <a href="#">Merken</a> |
| IT-Supporter First Level      | St. Gallen | 65000 - 78000 CHF   | employer    | <a href="#">Details</a> <a href="#">Merken</a> |
| Mobile Entwickler iOS         | Lausanne   | 90000 - 110000 CHF  | employer    | <a href="#">Details</a> <a href="#">Merken</a> |
| Cloud Architect               | Zug        | 120000 - 145000 CHF | employer    | <a href="#">Details</a> <a href="#">Merken</a> |

Items per page: 10 1 - 8 of 8 < > >|

#### Stellendetail

[← Zurück](#)

### Frontend Entwicklerin Angular

Bern · 85000 - 100000 CHF  
Arbeitgeber: employer  
Erstellt am 26.08.2026

#### Beschreibung

Für den Ausbau unserer Weboberflächen suchen wir Verstärkung im Frontend. Sie entwickeln mit Angular und TypeScript, arbeiten eng mit unserem UX-Team zusammen und bringen eigene Ideen in die Gestaltung ein.

[➤ Jetzt bewerben](#) [🔖 Job merken](#)

Bewerbung eingereicht

Schliessen

Stelle gemerkt

Schliessen

## Meine Bewerbungen

| Stelle                                 | Arbeitgeber | Status | Beworben am | Aktionen                                                                                         |
|----------------------------------------|-------------|--------|-------------|--------------------------------------------------------------------------------------------------|
| Frontend Entwicklerin Angular          | employer    | Offen  | 04.09.2026  |  Zurückziehen |
| Items per page: 10 1 - 1 of 1  < < > > |             |        |             |                                                                                                  |

## Gemerkte Jobs

| Stelle                 | Ort    | Gehalt             | Arbeitgeber | Gemerkt am | Aktionen                                                                                                                                                                                  |
|------------------------|--------|--------------------|-------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Systemadministrator    | Luzern | 75000 - 90000 CHF  | employer    | 04.09.2026 |  Zum Job  Entfernen |
| Senior Java Entwickler | Zürich | 95000 - 115000 CHF | employer    | 04.09.2026 |  Zum Job  Entfernen |
| Items per page:        |        |                    |             |            | <div><div>10</div><div></div></div>                                                                                                                                                       |
|                        |        |                    |             |            | 1 - 2 of 2                                                                                                                                                                                |
|                        |        |                    |             |            | <div> &lt; &lt; &gt; &gt; </div>                                                                                                                                                          |

## Mein Profil

Name\*

Bawan MAHMUD

12/100

E-Mail\*

bawan@bawan.ch

14/150

Rolle

JOB\_SEEKER

Die Rolle kann nicht selbst geändert werden

Speichern

Änderungen gespeichert

Schliessen

## 4. Arbeitgeber

1. Über „Meine Inserate“ die eigenen Stellen öffnen.
2. „Neue Stelle erfassen“ wählen und das Formular ausfüllen. Solange Eingaben fehlen oder ungültig sind, bleibt „Speichern“ gesperrt und die betroffenen Felder zeigen einen Hinweis.
3. Nach dem Speichern erscheint das Inserat in der Liste.
4. Über „Bearbeiten“ lässt sich ein bestehendes Inserat ändern, über „Löschen“ nach Rückfrage entfernen.
5. Über „Bewerbungen“ die eingegangenen Bewerbungen zu einem Inserat öffnen.
6. Eine offene Bewerbung nach Rückfrage annehmen oder ablehnen. Der geänderte Status ist unmittelbar beim Arbeitssuchenden unter „Meine Bewerbungen“ sichtbar.

## Dashboard

**Willkommen employer - (employer@employer.com)!**

 Stellen durchsuchen

 Meine Inserate

 + Neue Stelle erfassen

### Stellen

| + Neue Stelle erfassen        |        |                    |             |                                                                                               |
|-------------------------------|--------|--------------------|-------------|-----------------------------------------------------------------------------------------------|
| Titel                         | Ort    | Gehalt             | Arbeitgeber | Aktionen                                                                                      |
| Frontend Entwicklerin Angular | Bern   | 85000 - 100000 CHF | employer    |  Details |
| Data Analyst                  | Zürich | 88000 - 105000 CHF | emplyer     |  Details |

### Neue Stelle erfassen

 Zurück

Titel\*

Titel ist erforderlich

Beschreibung\*

hello

Beschreibung muss mindestens 10 Zeichen lang sein

Ort\*

Ort ist erforderlich

Gehaltsspanne

vt. 9franken

Format z.B. 90000 - 110000 CHF

Speichern

### Neue Stelle erfassen

 Zurück

Titel\*

Job

3/100

Beschreibung\*

Ein Job der Jobbt.

18/2000

Ort\*

Jobort

6/100

Gehaltsspanne

optional, z.B. 90000 - 110000 CHF

Speichern

## Meine Inserate

|                               |        |                    |            |                            | <a href="#">+ Neue Stelle erfassen</a>              |
|-------------------------------|--------|--------------------|------------|----------------------------|-----------------------------------------------------|
| Titel                         | Ort    | Gehalt             | Erstellt   | Aktionen                   |                                                     |
| Frontend Entwicklerin Angular | Bern   | 85000 - 100000 CHF | 26.08.2026 | <a href="#">Bearbeiten</a> | <a href="#">Bewerbungen</a> <a href="#">Löschen</a> |
| Data Analyst                  | Zürich | 88000 - 105000 CHF | 16.08.2026 | <a href="#">Bearbeiten</a> | <a href="#">Bewerbungen</a> <a href="#">Löschen</a> |

## Bewerbungen

|                 |                   |            |             |                            | <a href="#">← Zurück</a> Frontend Entwicklerin Angular                                                              |
|-----------------|-------------------|------------|-------------|----------------------------|---------------------------------------------------------------------------------------------------------------------|
| Bewerber        | E-Mail            | Status     | Beworben am | Aktionen                   |                                                                                                                     |
| seeker          | seeker@seeker.com | Angenommen | 27.08.2026  | —                          |                                                                                                                     |
| seeker          | seeker@seeker.com | Abgelehnt  | 31.08.2026  | —                          |                                                                                                                     |
| seeker          | seeker@seeker.com | Offen      | 03.09.2026  | <a href="#">✓ Annehmen</a> | <a href="#">✗ Ablehnen</a>                                                                                          |
| Bawan           | bawan@bawan.com   | Angenommen | 03.09.2026  | —                          |                                                                                                                     |
| Bawan MAHMUD    | bawan@bawan.ch    | Offen      | 04.09.2026  | <a href="#">✓ Annehmen</a> | <a href="#">✗ Ablehnen</a>                                                                                          |
| Items per page: |                   |            |             |                            | 10 <a href="#">1 - 5 of 5</a> <a href="#"> &lt;</a> <a href="#">&lt;</a> <a href="#">&gt;</a> <a href="#">&gt; </a> |

## 5. Administrator

- Über „Benutzer“ die Benutzerverwaltung öffnen. Sie listet alle registrierten Benutzer mit Rolle und Erstellungsdatum.
- Einen Benutzer nach Rückfrage entfernen.
- Über „Alle Bewerbungen“ die globale Übersicht öffnen. Sie zeigt sämtliche Bewerbungen aller Benutzer mit Stelle, Arbeitgeber und Bewerber.
- Auch hier lassen sich offene Bewerbungen annehmen oder ablehnen.

Administratoren haben bewusst keine eigenen Inserate, Bewerbungen oder Merklisten. Die entsprechenden Einträge fehlen in ihrer Navigationsleiste.

### Benutzerverwaltung

| Name                                                                                                                                | E-Mail                   | Rolle      | Erstellt   | Aktionen                |
|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------|------------|------------|-------------------------|
| seeker                                                                                                                              | seeker@seeker.com        | JOB_SEEKER | 27.08.2026 | <a href="#">Löschen</a> |
| employer                                                                                                                            | employer@employer.com    | EMPLOYER   | 31.08.2026 | <a href="#">Löschen</a> |
| Test Bewerber                                                                                                                       | test-bewerber@example.ch | JOB_SEEKER | 03.09.2026 | <a href="#">Löschen</a> |
| Bawan                                                                                                                               | bawan@bawan.com          | JOB_SEEKER | 03.09.2026 | <a href="#">Löschen</a> |
| Claude Test                                                                                                                         | claudettest@example.ch   | JOB_SEEKER | 04.09.2026 | <a href="#">Löschen</a> |
| Bawan MAHMUD                                                                                                                        | bawan@bawan.ch           | JOB_SEEKER | 04.09.2026 | <a href="#">Löschen</a> |
| Items per page: 10 <a href="#">1 - 6 of 6</a> <a href="#"> &lt;</a> <a href="#">&lt;</a> <a href="#">&gt;</a> <a href="#">&gt; </a> |                          |            |            |                         |

## Alle Bewerbungen

| Stelle                        | Arbeitgeber | Bewerber | Status     | Beworben am | Aktionen              |
|-------------------------------|-------------|----------|------------|-------------|-----------------------|
| Frontend Entwicklerin Angular | employer    | seeker   | Angenommen | 27.08.2026  | —                     |
| Data Analyst                  | employer    | seeker   | Offen      | 22.08.2026  | ✓ Annehmen ✗ Ablehnen |
| Senior Java Entwickler        | employer    | seeker   | Offen      | 29.08.2026  | ✓ Annehmen ✗ Ablehnen |

## 6. Rollenabhängige Anzeige und Zugriffsschutz

1. Ohne Anmeldung eine geschützte Adresse aufrufen, beispielsweise `/users`. Die Anwendung leitet auf die Seite „Kein Zugriff“ um.
2. Denselben Aufruf als angemeldeter Benutzer mit einer nicht berechtigten Rolle wiederholen. Das Ergebnis ist dasselbe.
3. Dieselbe Stellendetailseite nacheinander als Arbeitssuchender und als Arbeitgeber öffnen. Die Angaben zur Stelle sind identisch, die Aktionsleiste unterscheidet sich.

## 10. Testing

Für die Unit-Tests wird Vitest zusammen mit dem Angular Testing Framework eingesetzt. Ausgeführt werden die Tests mit `npm test`.

### Testdatei

**job-posting.service.spec.ts**

**job-list.component.spec.ts**

**registration.service.spec.ts**

**app-header.component.spec.ts**

### Geprüft

Alle Methoden des Services mit vollem CRUD-Umfang

Alle Methoden der Stellenlisten-Komponente

Registrierung inklusive der Fehlerfälle 409 und 400

Erzeugung der Kopfzeilen-Komponente

Die Service-Tests arbeiten mit dem `HttpTestingController`. Sie prüfen für jede Methode die aufgerufene Adresse, die HTTP-Methode und die Verarbeitung der Antwort. Nach jedem Test wird zusätzlich sichergestellt, dass keine unerwarteten Requests abgesetzt wurden.

Der Komponententest deckt alle Methoden der Stellenlisten-Komponente ab: das Laden der Liste, die Verbindung der Paginierung, beide Navigationsmethoden sowie das Markieren einer Stelle.

Insgesamt umfasst die Testsuite 19 Tests. Ergänzend wird der Codestil mit ESLint geprüft; die Regeln entsprechen denen des Demoprojekts.



---

## Fazit

Mit diesem Projekt wird eine vollständige Backend-Lösung für eine Jobplattform entwickelt, welche typische Funktionen eines modernen Bewerbungs- und Stellensystems abbildet. Der Fokus liegt auf einer sauberen Architektur, sicheren Authentifizierung sowie einer strukturierten REST-API.

Mit dem Frontend aus Modul 294 ist aus der REST-API eine vollständig bedienbare Anwendung geworden. Die im Backend angelegten Rollen wirken sich nun unmittelbar auf die Oberfläche aus: Jede Benutzergruppe sieht ausschliesslich die für sie vorgesehenen Seiten und Aktionen. Die Trennung zwischen Backend und Frontend hat sich dabei bewährt — bis auf die Auflösung des angemeldeten Benutzers und die Aktivierung des CSRF-Schutzes waren keine Eingriffe in die bestehende Fachlogik nötig.